

Architectural Trade-offs in Scientific Text Classification: A Comparative Study of Transformers and Deep Baselines

Guhan Elangovan

Student ID: 2600240455-8

Course: Artificial Intelligence

Ritsumeikan University

January 13, 2026

Abstract

The publication of scientific literature in fields such as bio-medical and material science has grown exponentially throughout the years. While, comprehensively understanding these articles through language processing methods has been done using several traditional baselines and generative Large Language Models (LLMs). Recent scientific publications with advanced and more complex semantic structures require highly precise domain adaptive deep-models which many traditional baselines fail to capture and, the ones who do are accompanied with heavy computational cost for its performance. The computational demand for the performance of these models have not been compared and analyzed. This study evaluates different deterministic model architectures such as SciBERT, TextCNN and Bi-LSTM on a scientific domain adaptation task to classify different abstracts from bio informatics, neuroscience, and material science to determine their competency and feasibility for their performance cost and efficacy in the given task. We compiled balanced datasets from springer and arXiv with a total of 1860 publicly available abstracts specifically curated for the three scientific domains. Experimentation results indicate that SciBERT significantly outperforming the other deterministic models with a classification accuracy of 98.71% on the Springer dataset and 91.18% on the arXiv dataset with a performance margin of over 50% compared to traditional baselines on generalization tasks. These findings suggests, while baseline models are good at fast interpretation, deep learning architectures such as fined-tuned domain-specific transformers outclass them in precision and accuracy with little performance and time overhead while acting as an efficient surrogate to advanced generative Large Language Models which are often studied to be performing worse in deterministic tasks such as text classification and large scale bibliography sorting. The source code and datasets are available at: https://github.com/Hangqbt/AI_Final_SciNlpLM

1 Introduction

The recent hike in publication volume across specialized scientific fields such as bio-informatics, neuroscience and material science has scaled to an unmanageable rate rendering it nearly impossible for an institution or a researcher to manually capture the whole picture of a particular domain. Recent surveys indicate, that millions of papers are published annually with almost 2 million articles released on platforms such as arXiv on 2022 leading to 36 million papers being released on that year alone globally [3]. With the sheer volume of scholarly articles it is no longer feasible to manually sort and categorize research papers, creating a need for a highly precise, automated systems for bibliography classification tasks.

Scientific abstracts from research articles contain advanced and domain specific jargon and polysemous words that traditional and general language models fail to grasp. Similarly, models trained on general texts such as wikipedia often lack the contextual embeddings to identify the nuances between fields such as bio-informatics and neuroscience. This lack of understanding leads to misclassification which ultimately hinders the systematic reviews and the meta-analyses.

Large Language Models (LLMs) have been in the rise since the introduction of the transformers [7] and their zero shot performance in the Natural Language Processing (NLP) domain has made it a primary player and a dominant approach. However specific classification tasks causes the LLMs to suffer from high computational costs with sub-optimal performance, lack of reproducibility and its struggle with domain specific vocabulary has made it unsuitable with its current efficiency metrics [6]. This leaves a gap regarding the deterministic architectures that balance higher accuracy rates with high computational efficiency.

This report addresses the gap by evaluating three distinct deterministic architectures, namely, SciBERT (a domain-adaptive transformer) [1], TextCNN (a convolutional text processing baseline) [5] and finally, Bi-LSTM (a sequential baseline) [4] in a comparative study to demonstrate the optimal method for domain specific text processing targeted towards classification sorting applications. This literature includes the use of two curated datasets from containing abstracts from Springer and arXiv to ensure robust testing across peer reviewed and pre-print standards of academic literature. The main objectives for this report is as follows,

- To benchmark the accuracy of different model architecture under scientific text-based classification task.
- To test cross dataset generalization across all models.
- To demonstrate that fine-tuned models with minimal size can be the 'SOTA' (State of the Art) for the task in hand.

2 Literature Review

Early text classification heavily relied on feature engineering methods such as TF-IDF and Bag-of-Words which measured word frequency but completely missed it's contextual meaning in the sentence. However,

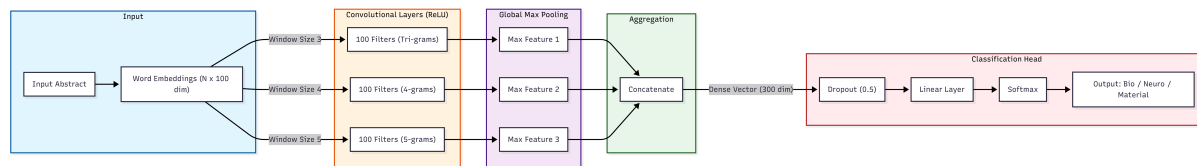


Figure 1: Architecture of a Convolutional Neural Network (TextCNN) for sentence classification

the shift to technologies based on neural networks with models like Word2Vec has allowed semantic relationships between words to be interpreted during processing. The introduction of Convolutional Neural Networks, primarily used in image processing applications introduced by [5], acted as a breakthrough for capturing local patterns (n-grams) in a more efficient manner. Similarly, the formulation of the LSTMs [4] with their ability to consistently hold long-range dependencies in sequential texts was an approach that was utilized before the introduction of transformer based models. However, these models did come with own flaws and limitations such as limited context understanding and long training times.

The invention of the transformer architecture [7], which replaced sequential processing with methods like "Self-Attention", allowing the models to process context at a much larger scale. With further development by other scholars, Bidirectional Encoder Representations for Transformers (BERT) [2], a model under the transformer family, was deployed with the ability to process texts from both ends of the sentence (i.e, from left to right and right to left) to understand the context in a deeper sense. However, while BERT was a powerful starting point for future developments, it was trained on wikipedia and BookCorpus data making it suboptimal in highly scientific use cases.

Processing scientific texts with high context sensitivity requires "Domain-Adaptive Pre-Training" (DAPT) for the model to understand and distinguish between the context of a word between fields. For Example, the context of the phrase "Neural Network" between biology and computer science vary significantly however, it requires extensive knowledge of the field and a better understand of the surrounded context to defined the definition of the phrase in hand. Following this, [1] introduced SciBERT, a model that utilized the same transformer architecture but was pre-trained on corpus of 1.14 million scientific publications from semantic scholars. SciBERT specifically uses domain-specific vocabulary (SCIVOCAB) to reduce token fragmentation for scientific terminologies compared to traditional BERT models.

Recent studies highlight the increased use of LLMs in generative tasks including NLP Applications [3]. However, Discriminative tasks such as classifications are often done poorly by LLMs which introduce massive computational and stochastic overheads along with hallucinations and a lack of explainability of its real-time process (black box) [6]. On the other hand, smaller and deterministic models such as the BERT family, yields reproducible results that are often superior to "Zero-shot" generative outputs of LLMs for domain specific tasks. Therefore, this study focuses on comparing these deterministic architectures to determine a possible surrogate to solve the bibliography sorting problem in a more efficient and automated manner.

3 Methodology

3.1 Data Acquisition

The data used in the trials were acquired from two distinct sources, namely, Springer Nature and ArXiv to cover peer reviewed and pre-print forms of scientific publications. The publicly available abstracts were fetched and stored in an automated fashion over multiple requests to the sites mentioned above. Three target labels were chosen for the creation of the dataset, specifically, bio-informatics, neuroscience and material science. These topics were selected since they share overlapping terminologies, for example, "Network" or "cells", making classification generally harder, challenging and less probable for the model to learn quickly and overfit in general. The final dataset that was curated consists of 1860 abstracts (930 from Springer and 930 from ArXiv) with exactly 310 from each class per source to eliminate bias from class imbalance.

3.2 Data Pre-processing

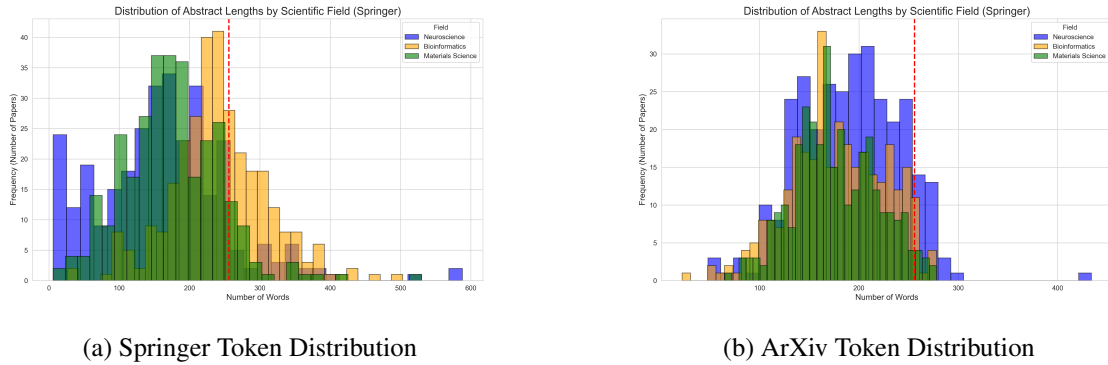


Figure 2: **Word Distribution Graphs** (a) Springer Dataset Distribution (b) arXiv Dataset Distribution.

The data, once acquired, was cleaned by converting all the abstracts to lowercase followed by the removal of HTML tags and finally the stripping of non-alphanumeric character except for scientific notations. This workflow was fully automated to process both the datasets. After cleaning, the token count distribution across the datasets were measured to establish the optimal maximum sequence length to be used for the models. The distribution of the Springer dataset suggested that 81.51% of the dataset were within 256 words while the remaining 18.49% of the papers had abstract lengths over 256. Similarly, Around 4.52% of the ArXiv dataset would be over the word count of 256. hence, the maximum sequence length was determined to be 256 to improve speed while retaining minimal losses in context clues. The tokenization approach varied for between the models, for SciBERT, WordPiece tokenizer was used which helped preserve sub word information such as "neuro" + "science" while the other baselines used, standard white space tokenization which was then built into a custom vocabulary.

3.3 Model Architectures

3.3.1 SciBERT (Fine-Tuned)

The SciBERT Model was extracted from `allenai/scibert-scivocab-uncased` from HuggingFace to be used as the feature extractor. The input to the model is formatted as "[CLS] + Abstract + [SEP]",

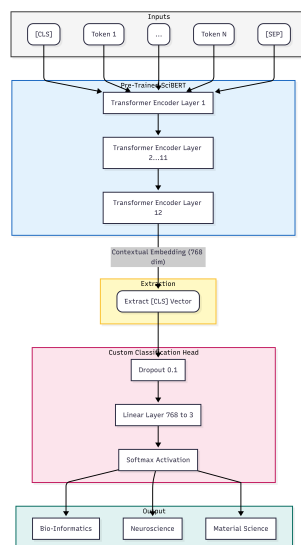


Figure 3: The Fine-Tuned SciBERT Architecture

where the final hidden state of the CLS token holds the aggregated context and understanding of the text with the help of self attention mechanisms. In addition to this, the custom classification head in the model, has a dropout layer for better regularization followed by a Linear layer that maps to a base architectural output of 768 dimensional hidden units [1] with 3 output classes for each topic label.

3.3.2 TextCNN

The TextCNN Model interprets the sentences like images using an one dimensional convolution filter which slides over the sentence. The model is implemented with three parallel convolutional layers with kernel sizes of 3, 4 and 5 which essentially allows the model to capture local semantic context and features. Each kernel size utilized 100 distinct filters which resulted in an extraction which had a diverse pattern of n-grams. Finally to extract the strongest features from the feature maps, a global max-pooling was applied to the output of each convolutional layer ultimately making the model more resistant to difference in positions of key terminology.

3.3.3 Bi-LSTM

The Bidirectional Long Short-Term Memory network is another baseline that is tested to capture the sequential dependencies of the abstracts. It processes the texts in two directions, namely, forward and backward. Resulting in a better temporal understand of the text when applicable. The final classification vector is defined by concatenating all the final hidden states from both directions to create a dense representation that summarizes the entire context of the abstract being processed.

4 Exploratory Data Analysis (EDA)

4.0.1 Class Distribution

Figure 4 represents the class distribution between the two sources. As mentioned in the methodology, the datasets were created with a balanced count of 310 samples per topic label to create a uniform distribution

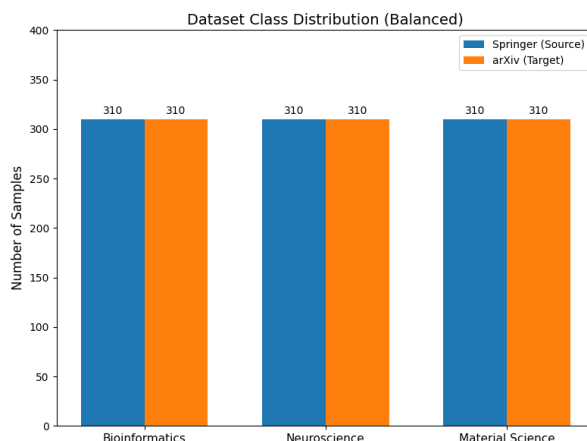


Figure 4: Label Class Distribution

and prevent any disturbances in the results due to majority class based biases. This curation process was followed for both datasets that were created.

4.0.2 Vocabulary Analysis

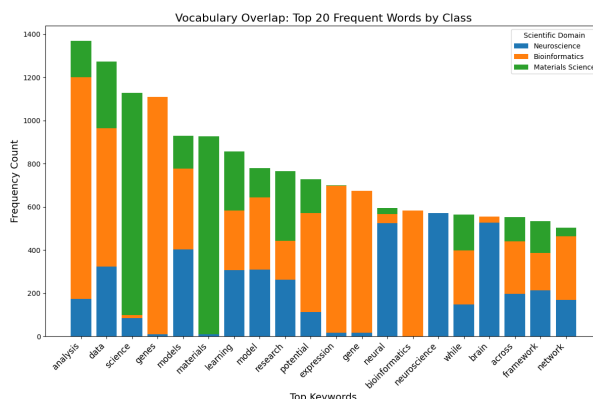


Figure 5: Vocabulary Overlap Analysis

To understand and measure the semantic overlap between domains, an analysis to find the 20 most frequently used words across the corpus was done. As hypothesized, words such as "models", "data", "analysis" and "learning" appear frequently among the data across all three categories. This high degree of overlap in lexical usage confirm that simple keyword matching approaches will be insufficient and would probably require some level of contextual understanding which are often found in deep learning architectures such as transformers.

5 Experimental Setup

5.1 Reproducibility

To ensure high reliability and determinism of the inference results from the models, the randomness present throughout the code and its various stochastic components are strictly locked down using a fixed seed to maintain consistency across all the model implementations. The random seed was set to 42 for the Python's

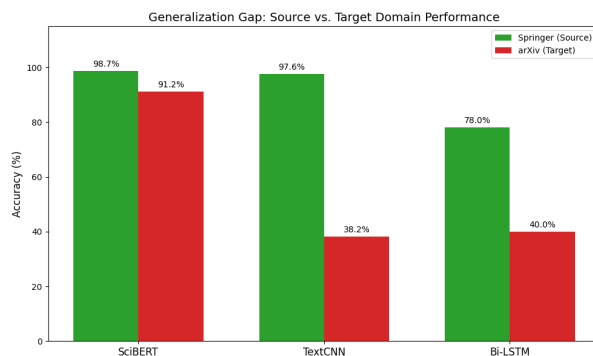


Figure 6: Generalization Gap

inbuilt random library, Numpy, Pytorch and the HuggingFace tokenizer. Finally, CUDA Algorithms were also forced to be locked to be deterministic (`torch.backends.cudnn.deterministic = True`). While forcing this to be deterministic does come with some performance overhead, it is considered a good trade off for guaranteed identical weight initializations and data shuffling for every run.

5.2 Validation Approach

The primary benchmark experiments which were run used a stratified 5-fold Cross-validation on the Springer dataset. The dataset was stratified to maintain a balance between the samples split between the classes and eliminate a local imbalance when the dataset is divided for the fold during training and testing. Finally to evaluate the robustness and generalization of the models, the optimal checkpoints of each architecture were evaluated on the unseen, noisy, ArXiv dataset. This essentially measures each model's performance on interpreting and generalizing different writing styles and context shifts in domains without needing any additional training.

5.3 Implementation and Hyper Parameters

All the models that were used during experimentation were implemented using the PyTorch and the Hugging face Transformers library along with Scikit-learn for data splitting and metric calculation. Both the baseline models (TextCNN and Bi-LSTM) were trained with 8 epochs with a batch size of 32 under a learning rate of $1e-3$ along with the help of Adam optimizer and Cross-Entropy loss function. On the other hand, SciBERT was fine-tuned with its existing weights and a batch size of 16 for 3 epochs under a default learning rate of $5e-5$ and weight decay of 0.01. Additionally, the input was processed with a WordPiece tokenizer and optimized with the help of the AdamW optimizer (adam with weight decay) to improve its memory and reduce the loss of context over long runs. Finally, all experiments were conducted on a RTX 3060 (Laptop) GPU to ensure consistent inference times and results.

6 Results and Discussion

6.1 Comparative Performance

In Table 1, introduces the results of the study conducted with the three model architectures on the classification task. It summarizes mean accuracy, the F1-score and the total training time across the

Table 1: Comparative performance metrics (Mean $\pm$ Std) using 5-Fold Cross-Validation on the Springer dataset. Training time represents the total duration for 8 epochs (Baselines) or 3 epochs (SciBERT).

Model	Accuracy (%)	F1-Score	Training Time (s)
SciBERT	98.71 $\pm$ 0.82	0.987	$\sim$ 154
TextCNN	97.63 $\pm$ 0.90	0.976	$\sim$ 358
Bi-LSTM	77.96 $\pm$ 3.16	0.778	$\sim$ 1.7

5 stratified folds done in all the three architectural types. From the results, we infer that SciBERT is the clear winner out of all the three models achieving the highest performance with a mean accuracy of 98.71% and an F1-score of 0.987, this level of accuracy demonstrates near-perfect performance of classifying the springer dataset and shows great potential on generalization ability of the model. While SciBERT Achieved the best score, TextCNN, maintained a competitive accuracy of 97.63% enforcing that convolution based interpretation is a sufficient and successful approach for understanding scientific contexts and lexicons. In contrasts to these models, Bi-LSTM, performed the poorest with a prediction accuracy of 77.96% suggesting that traditional sequential modeling approaches struggle to contextualize a meaning without attention mechanisms when processing long-range dependencies in complex scientific abstracts. However, during training, Bi-LSTM was nearly instant with around 1.7 seconds of total training time when TextCNN surprisingly consuming around 358 seconds and SciBERT consuming around 154 seconds. This result definitively show that SciBERT is not only the most accurate model but also the best in terms of efficiency to performance and convergence speeds (3 epochs for SciBERT and 8 epochs for baselines) compared to the baselines.

6.2 Cross-Dataset Generalization

Table 2: Cross-dataset generalization performance. Models trained on Springer were evaluated on the arXiv dataset to measure their performance against unseen data.

Model	arXiv Accuracy (%)	Performance Drop	Verdict
SciBERT	91.18	-7.5%	Robust
Bi-LSTM	40.00	-37.9%	Failed
TextCNN	38.17	-59.5%	Failed

Table 2, introduces the results of the ArXiv dataset inference test which highlight significant performance variations amongst the models in terms of generalization on unseen datasets. Baseline models such as TextCNN and Bi-LSTM has suffered from a significant loss in prediction accuracy down to 38.17% and 40.00% respectively. The probability for a random guess on the dataset is 33.33% and the models performance is only a slight bit higher indicating that the baselines have essentially memorized the springer vocabulary rather than learning the generalized contexts behind the scientific terminology. In contrast, SciBERT, consistently maintained an accuracy of 91.18% on the unseen and noisy arXiv dataset proving its generalization capability and its conceptual understand of scientific terminology with minimal memorization. Furthermore, this gap in performance between models, with more than 50% when compared to SciBERT makes it the superior model with contextual embeddings allowing it to generalize different writing styles while deterministic baselines tend to overfit to the source domain and its contents.

6.3 Ablation Study

Table 3: Ablation study on SciBERT hyperparameters and architectural choices. The "Frozen Backbone" experiment significantly underperformed, demonstrating the necessity of fine-tuning. The "Deep Head" configuration achieved the highest single-run accuracy.

Experiment Configuration	LR	Batch	Backbone	Head Type	Accuracy (%)
Baseline (Standard)	2e-5	8	Fine-Tuned	Simple	98.92
Frozen Weights	1e-3	8	Frozen	Simple	90.86
High Learning Rate	5e-5	8	Fine-Tuned	Simple	99.46
Low Learning Rate	1e-5	8	Fine-Tuned	Simple	98.39
Large Batch Size	2e-5	16	Fine-Tuned	Simple	98.39
Deep Head + Dropout (0.3)	2e-5	8	Fine-Tuned	Deep	99.46

While SciBERT was the clear outlier amongst the models, its reliance on hyperparameters needed to be studied. The ablation study was done to observe the impact of fine-tuning the pre-trained weights of the model versus using it as a static extraction method without any modification from its base configuration. Initial tests with frozen backbones and training the classification head alone produced significant accuracy loss of around 8.06% with an inference percentage of 90.86% as compared to the fully fine-tuned result of 98.92%. Following this, adding a deeper head (4 layers) with a higher dropout rate of 0.3 yielded the absolute best performance from the model even seen from all the runs with a prediction percentage of 99.46% validating that additional refinement to the architecture definitely improves performance and generalization. While the pre-trained knowledge of the model is paramount, the task-specific fine-tuning is highly necessary for optimizing the additional 9% of performance seen in the results making it suitable for SOTA Applications.

6.4 Error Analysis & Discussion

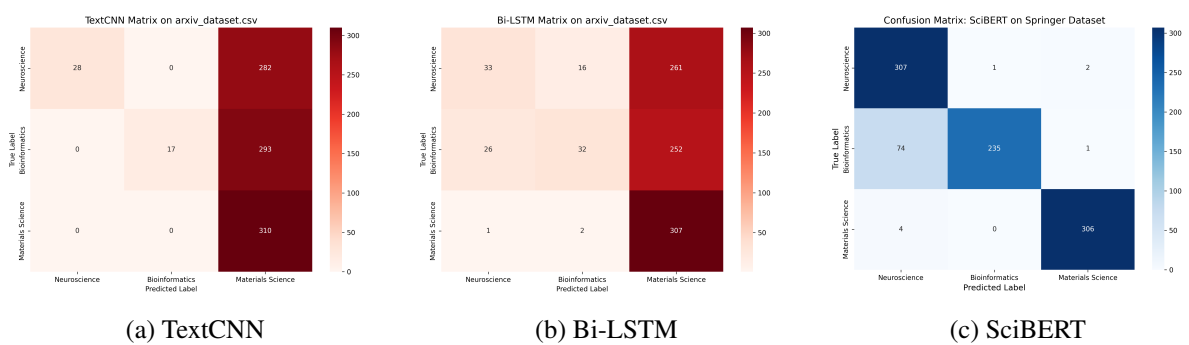


Figure 7: **Confusion Matrices on the Generalization (arXiv) Dataset.** (a, b) The baseline models , (c) SciBERT

During EDA we realized that, there are lexical overlapping between topic labels such as "analysis" and "model", this specific overlapping feature is highly likely to be a factor that may have caused models such as TextCNN which looks for specific words, to get confused when the writing styles shifted in the arXiv dataset. On the other hand, SciBERT, thanks to its attention mechanism, has allowed it to overcome this restriction and understand the context behind such words based on its surroundings, enabling it to

have high prediction accuracy even during a change in writing style and structure. Ultimately, for the task of scientific bibliography classification, the one-time training cost in terms of computational time and resources is negligible to the massive generalization performance it offers over traditional deep learning baselines.

The confusion matrices indicate the generalization gap that exists between the models. SciBERT as expected, exhibits consistent precision across all domains while the baseline models display significant off-diagonal clustering. To elaborate, we can visually see that "Neuroscience" is often misclassified as "Bioinformatics" by the traditional baseline models. This supports that attention mechanisms add great effect in distinguishing contexts despite the lexical overlap when the writing style changes.

7 Conclusion

The paper addressed the problem of automated scientific bibliography classification based sorting tasks that is required a high-performing surrogate model which is efficient enough to be suitable for SOTA Applications. The study evaluated three models from distinct deterministic architectures, specifically, TextCNN, Bi-LSTM and SciBERT. With the results of the study, sciBERT demonstrated superior performance and robustness across all experiments and datasets by maintaining an accuracy of 91.18% on the noisy, unseen ArXiv dataset whereas other baselines fell towards the 38-40% range suffering catastrophic drops in performance signifying sciBERT the superior architecture among them and indicating that the transformer architecture has incredible generalization capability in scientific text classification.

The stark contrast in generalization proves that traditional baselines such as TextCNN and Bi-LSTM rely heavily on keyword memorization causing them to overfit to the source and ultimately hindering their performance when the writing style or structure changes. On the other hand, SciBERT's domain-adaptive nature and its self attention mechanisms, enables it to interpret context of polysemous scientific terms such as "Model" and "Analysis" rather than purely relying on the frequency of appearance. Ultimately, with the results acquired from the tests, we can determine that, fine-tuned deterministic models offer a highly efficient and reproducible alternative to expensive LLMs for high volume sorting and classification tasks in complex domains.

Finally, a limitation of this study is its focus on three broad, widely known domains for analysis, namely, bio-informatics, neuroscience and material science. As future work, we propose that hierarchical classification into granular subfields could possibly improve model performance and prove practical utility. Finally, introducing interdisciplinary fields such as quantum biology could highlight and reveal the nuances, the potential robustness and detailed limitations of the model.

References

- [1] Iz Beltagy, Kyle Lo, and Arman Cohan. Scibert: A pretrained language model for scientific text. *arXiv preprint arXiv:1903.10676*, 2019.

- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [3] Xanh Ho, Anh Khoa Duong Nguyen, An Tuan Dao, Junfeng Jiang, Yuki Chida, Kaito Sugimoto, Huy Quoc To, Florian Boudin, and Akiko Aizawa. A survey of pre-trained language models for processing scientific text. *arXiv preprint arXiv:2401.17824*, 2024.
- [4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [5] Yoon Kim. Convolutional neural networks for sentence classification. *arXiv preprint arXiv:1408.5882*, 2014.
- [6] Zhyar Rzgar K Rostam and Gábor Kertész. Fine-tuning large language models for scientific text classification: A comparative study. In *2024 IEEE 6th International Symposium on Logistics and Industrial Informatics (LINDI)*, pages 000233–000238. IEEE, 2024.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.